

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №1
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин А.А.

Краснодар
2026

Тема. Градиентные методы оптимизации.

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение метода градиентного спуска с постоянным шагом для поиска минимума заданной функции.

Методы поиска с использованием производных. Пусть дана функция $f(x)$, ограниченная снизу на множестве R^n и имеющая непрерывные частные производные во всех его точках. Требуется найти локальный минимум функции $f(x)$ на множестве допустимых решений $X = R^n$, т.е. найти такую точку $x^0 \in R^n$, что

$$f(x^0) = \min_{x \in R^n} f(x).$$

Градиентные методы или методы первого порядка используют значения частных производных первого порядка целевой функции. Методы второго и более высоких порядков используют значения частных производных второго и более высоких порядков целевой функции. Рассмотрим метод градиентного спуска с постоянным шагом.

Стратегия решения задачи состоит в построении релаксационной последовательности точек $\{x^k\}$, $k = 0, 1, \dots$, таких что $f(x^{k+1}) < f(x^k)$. Точки последовательности $\{x^k\}$ вычисляются по правилу:

$$x^{k+1} = x^k - t_k \nabla f(x^k).$$

где t_k – величина шага спуска. Начальная точка x^0 задается исследователем. В качестве направления спуска выберем $-\nabla f(x^k)$ – антиградиент функции $f(x)$, вычисленный в точке x^k .

Качество методов оценивается их скоростью сходимости:

- 1) линейная скорость сходимости – скорость сходимости геометрической прогрессии: $\|x_{k+1} - x^*\| \leq q \cdot \|x_k - x^*\|$, $0 < q < 1$;
- 2) сверхлинейная скорость сходимости, если $\|x_{k+1} - x^*\| \leq q_k \cdot \|x_k - x^*\|$, где $q \rightarrow 0$ при $k \rightarrow \infty$;
- 3) квадратичная скорость сходимости: $\|x_{k+1} - x^*\| \leq c \cdot \|x_k - x^*\|^2$, где $c > 0$.

Метод градиентного спуска с постоянным шагом.

Стратегия поиска. Величина шага t_k задается пользователем и остается постоянной до тех пор, пока функция убывает в точках последовательности, что контролируется путем проверки выполнения условия $f(x^{k+1}) - f(x^k) < 0$ или $|f(x^{k+1}) - f(x^k)| < \varepsilon \|f(x^k)\|^2$, $0 < \varepsilon < 1$. Построение последовательности $\{x^k\}$ заканчивается в точке x^k , для которой $\|f(x^k)\| < \varepsilon_1$, где ε_1 – заданное малое положительное число; или $k \geq M$, где M – предельное число итераций; или при двукратном одновременном выполнении двух неравенств $\|x^{k+1} - x^k\| < \varepsilon_2$,

$|f(x^{k+1}) - f(x^k)| < \varepsilon_2$, где ε_2 – малое положительное число. Вопрос о том, может ли точка x^k рассматриваться как найденное приближение искомой точки минимума, решается путём проведения дополнительного исследования, которое описано ниже.

Алгоритм.

Шаг 1. Задать x , $0 < \varepsilon < 1$, $\varepsilon_1 > 0$, $\varepsilon_2 > 0$, M – предельное число итераций.

Найти градиент функции в произвольной точке $\nabla f(x) = \left(\frac{\partial f(x)}{\partial x_1}, \dots, \frac{\partial f(x)}{\partial x_n} \right)^T$.

Шаг 2. Положить $k = 0$.

Шаг 3. Вычислить $f(x^k)$.

Шаг 4. Проверить выполнение критерия окончания $|\nabla f(x^*)| < \varepsilon_1$:

- а) если критерий выполнен, то расчёт закончен и $x^* = x^k$;
- б) если критерий не выполнен, то перейти к шагу 5.

Шаг 5. Проверить выполнение неравенства $k \geq M$:

- а) если неравенство выполнено, то расчет окончен: $x^* = x^k$;
- б) если нет, то перейти к шагу 6.

Шаг 6. Задать величину шага t^k .

Шаг 7. Вычислить $x^{k+1} = x^k - t_k \nabla f(x^k)$.

Шаг 8. Проверить выполнение условия:

$f(x^{k+1}) - f(x^k) < 0$ (или $|f(x^{k+1}) - f(x^k)| < \varepsilon \|\nabla f(x^k)\|^2$);

- а) если условие выполнено, то перейти к шагу 9;
- б) если условие не выполнено, положить t_k и перейти к шагу 7.

Шаг 9. Проверить выполнение условий

$\|x^{k+1} - x^k\| < \varepsilon_2$, $\|f(x^{k+1}) - f(x^k)\| < \varepsilon_2$:

- а) если оба условия выполнены при текущем значении k и $k = k - 1$, то расчет окончен и $x^* = x^{k+1}$;
- б) если хотя бы одно из условий не выполнено, положить $k = k + 1$ и перейти к шагу 3.

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки ruqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

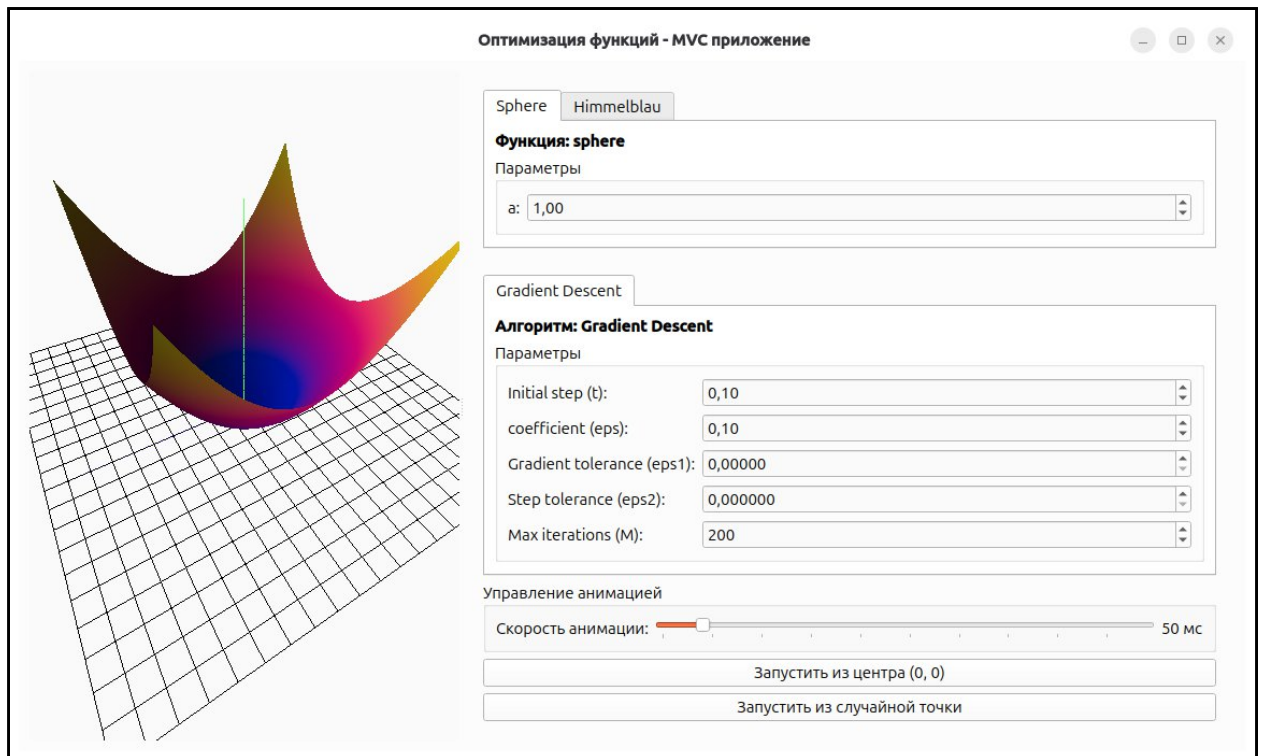


Рисунок 1 — Главное окно приложения

2. Доступные функции: Sphere и Himmelblau (рисунок 2).

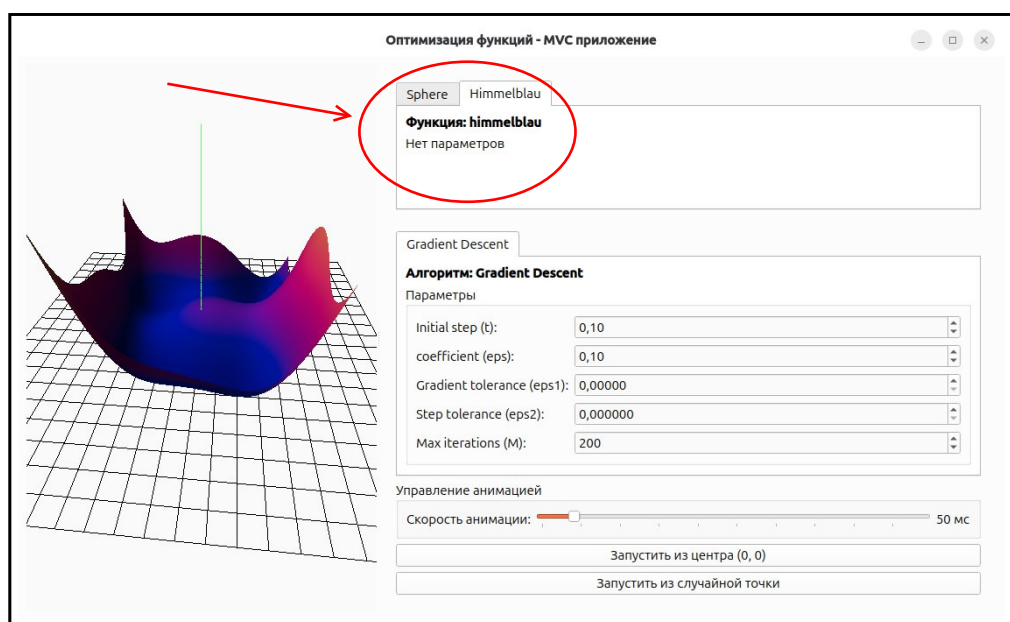


Рисунок 2 — Доступные функции

3. Доступный алгоритм оптимизации: градиентный спуск с постоянным шагом (рисунок 3).

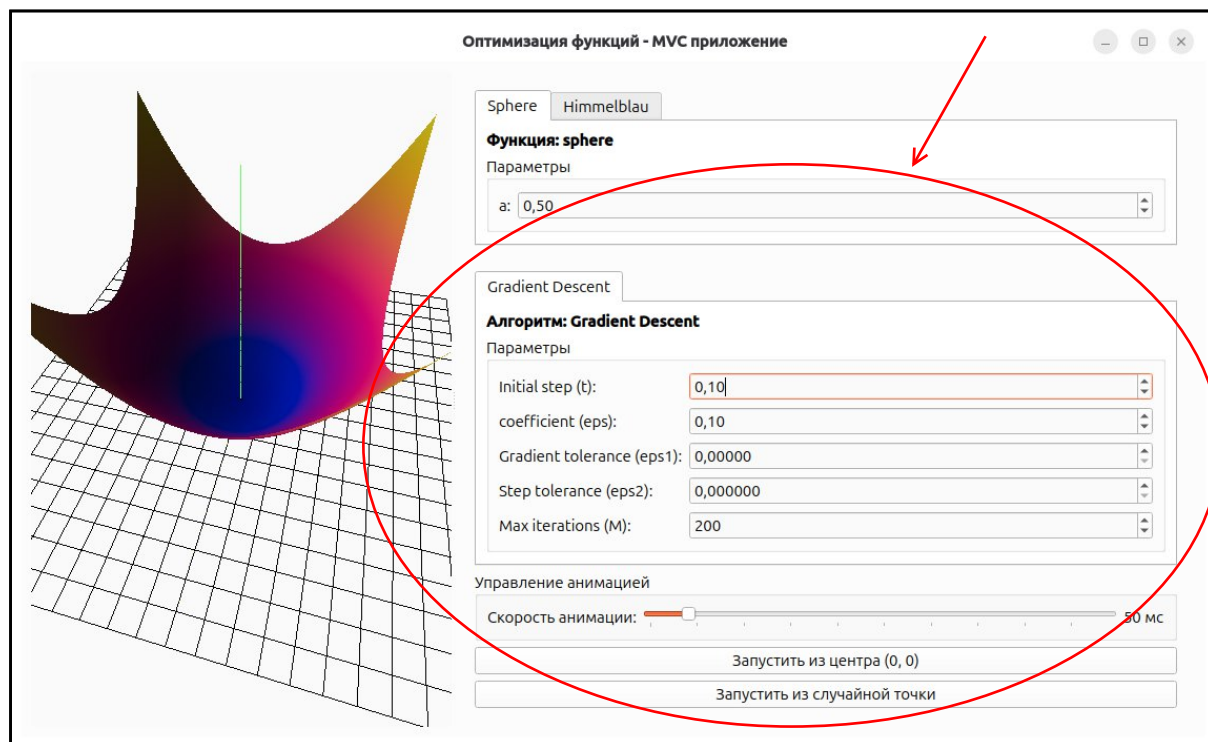


Рисунок 3 — Доступный алгоритм оптимизации

4. Запустим алгоритм поиска минимума функции Sphere из случайной точки (рисунок 4).

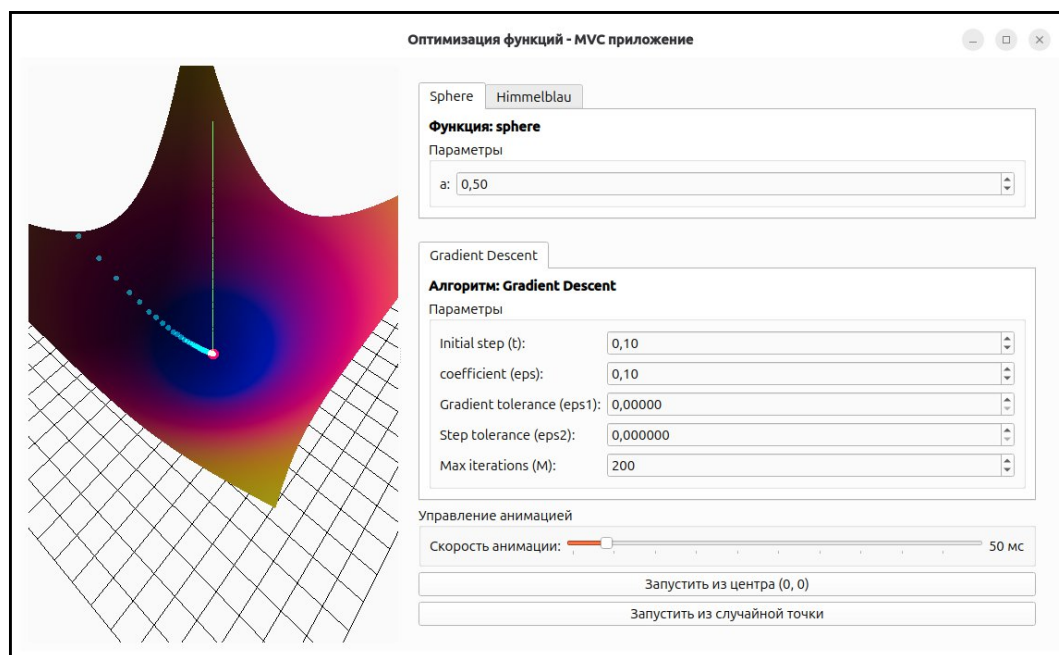


Рисунок 4 — Запуск алгоритма оптимизации

5. Запустим алгоритм поиска минимума функции Himmelblau из точки (0, 0) (рисунок 5).

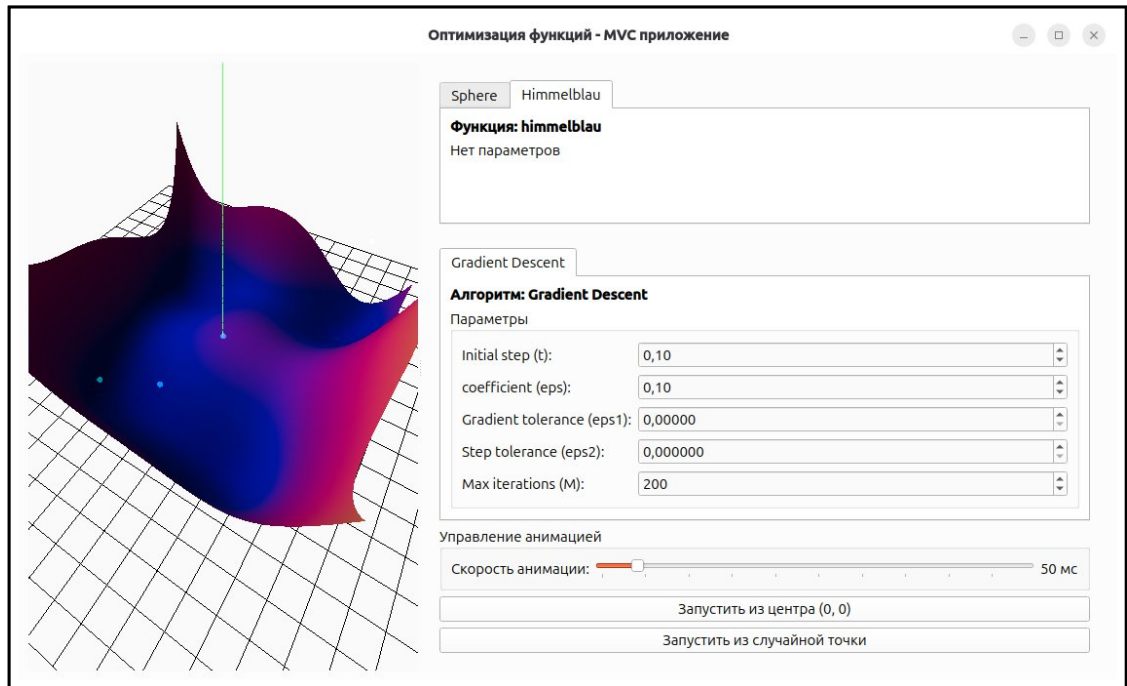


Рисунок 5 — Запуск алгоритма

6. Запустим алгоритм оптимизации с изменёнными параметрами (рисунок 6).

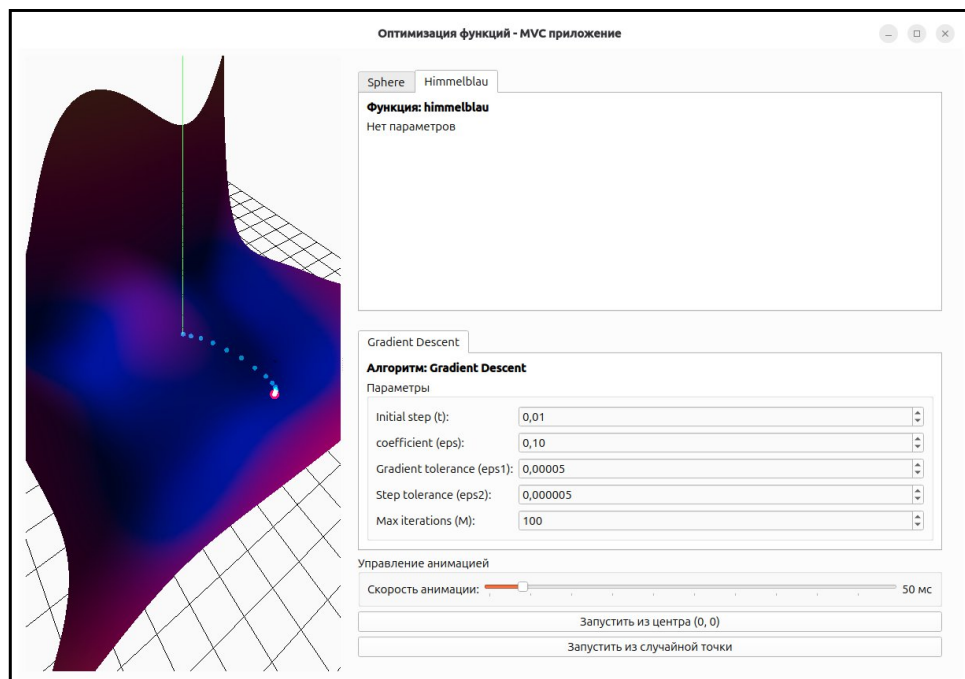


Рисунок 6 — Оптимизация с изменёнными параметрами

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение метода градиентного спуска с постоянным шагом для поиска минимума заданной функции.

Листинг программы.

```
from math import sqrt

class OptimizationAlgorithms:
    """Класс с алгоритмами оптимизации"""

    @staticmethod
    def gradient_descent(func, x0, y0, t, eps, eps1, eps2, M, **func_params):
        """
        Градиентный спуск с делением шага (backtracking)
        """

        iterations = int(M)
        k = 0
        h = 1e-5

        x, y = x0, y0
        yield (x, y)
        while k < iterations:
            grad_x = (func(x + h, y, **func_params) - func(x - h, y, **func_params)) / (2 * h)
            grad_y = (func(x, y + h, **func_params) - func(x, y - h, **func_params)) / (2 * h)

            grad_norm = sqrt(grad_x ** 2 + grad_y ** 2)

            if grad_norm <= eps1:
                return (x, y)
            # Backtracking
            t_k = t
            while True:
                tmp_x = x - t_k * grad_x
                tmp_y = y - t_k * grad_y

                f_current = func(x, y, **func_params)
                f_next = func(tmp_x, tmp_y, **func_params)

                if (f_next - f_current < 0) or \
                    (abs(f_next - f_current) < eps * grad_norm ** 2):
                    break
                else:
                    t_k /= 2

            step_norm = sqrt((tmp_x - x) ** 2 + (tmp_y - y) ** 2)
            func_diff = abs(f_next - f_current)

            # обновляем точку
            x_new, y_new = tmp_x, tmp_y

            yield (x_new, y_new)

            if step_norm < eps2 and func_diff < eps2:
                return (x_new, y_new)
            x, y = x_new, y_new
```

```

        k += 1

    return (x, y)

@staticmethod
def get_algorithm_params(algo_name):
    """Возвращает параметры алгоритма"""

    params = {
        'gradient_descent': [
            {
                'name': 't',
                'label': 'Initial step (t)',
                'default': 0.1,
                'min': 0.0001,
                'max': 1.0,
                'step': 0.01,
            },
            {
                'name': 'eps',
                'label': 'coefficient (eps)',
                'default': 0.1,
                'min': 0.0001,
                'max': 1.0,
                'step': 0.01,
            },
            {
                'name': 'eps1',
                'label': 'Gradient tolerance (eps1)',
                'default': 1e-4,
                'min': 1e-6,
                'max': 1e-1,
                'step': 1e-5,
            },
            {
                'name': 'eps2',
                'label': 'Step tolerance (eps2)',
                'default': 1e-6,
                'min': 1e-8,
                'max': 1e-2,
                'step': 1e-6,
            },
            {
                'name': 'M',
                'label': 'Max iterations (M)',
                'default': 200,
                'type': int,
                'min': 10,
                'max': 2000,
                'step': 10,
            },
        ],
    }
    return params.get(algo_name, [])

@staticmethod
def get_available_algorithms():
    """Возвращает список доступных алгоритмов"""
    return ['gradient_descent']

```